

POLITECHNIKA WROCŁAWSKA	
Wydział Informatyki i Telekomunikacji	
Algorytmy i złożoność obliczeniowa	
Rok akademicki:	2024/2025
Autor projektu:	Kacper Piekut
Nr indeksu:	280879
Temat projektu:	Zadanie 2: Badanie efektywności algorytmów grafowych w zależności od rozmiaru instancji i sposobu pamiętania grafu

Oświadczenie: Przekazując to sprawozdanie do oceny prowadzącemu zajęcia Autor oświadcza, że zostało ono przygotowane samodzielnie, bez udziału osób trzecich oraz że żadna jego część nie jest plagiatem.

1. Wstęp

Grafy są jedną z podstawowych struktur danych wykorzystywanych w informatyce, matematyce dyskretniej oraz w wielu dziedzinach inżynierii. Pozwalają one modelować relacje i powiązania pomiędzy obiektami, co znajduje zastosowanie m.in. w analizie sieci komputerowych, optymalizacji tras w transporcie, projektowaniu układów elektronicznych czy analizie sieci społecznościowych. W teorii grafów szczególne znaczenie mają algorytmy wyznaczania najkrótszych ścieżek, które umożliwiają znalezienie optymalnych tras pomiędzy wybranymi wierzchołkami grafu. Jednym z najbardziej znanych i szeroko stosowanych algorytmów tego typu jest algorytm Dijkstry. Pozwala on na wyznaczenie najkrótszej ścieżki z jednego wierzchołka do wszystkich pozostałych w grafie o nieujemnych wagach krawędzi. Jego uniwersalność i względna prostota sprawiają, że jest on wykorzystywany zarówno w aplikacjach teoretycznych, jak i praktycznych, takich jak systemy nawigacyjne czy protokoły routingu w sieciach komputerowych. W literaturze teoretycznej przyjmuje się, że dla grafów rzadkich (o niewielkiej liczbie krawędzi w stosunku do liczby możliwych połączeń) lista sąsiedztwa pozwala na szybsze wykonywanie operacji związanych z przeglądaniem sąsiadów wierzchołka, co przekłada się na lepszą wydajność algorytmów grafowych, w tym algorytmu Dijkstry. Z kolei dla grafów gęstych, gdzie liczba krawędzi jest zbliżona do maksymalnej możliwej, przewaga listy sąsiedztwa maleje, a prostota i lokalność pamięciowa macierzy sąsiedztwa może okazać się korzystna. W niniejszym projekcie analizowana jest praktyczna wydajność algorytmu Dijkstry w zależności od wybranej reprezentacji grafu oraz parametrów takich jak liczba wierzchołków i gęstość grafu.

2. Cele projektu

- Zaimplementowanie algorytmu Dijkstry dla reprezentacji listowej oraz macierzowej grafu w języku C++.
- Przeprowadzenie serii eksperymentów dla grafów o różnych rozmiarach i gęstościach, generowanych losowo.
- Analiza uzyskanych wyników oraz sformułowanie wniosków dotyczących efektywności obu podejść w zależności od charakterystyki grafu.

3. Plan eksperymentu

Algorytm do testowania

Testowany jest klasyczny algorytm Dijkstry, zaimplementowany w wersji z kolejką priorytetową (min-kopcem), zarówno dla grafu reprezentowanego jako lista sąsiedztwa (dynamiczna tablica wektorów par), jak i jako macierz sąsiedztwa (tablica dwuwymiarowa). W obu przypadkach algorytm wyznacza najkrótszą ścieżkę pomiędzy losowo wybranymi wierzchołkami grafu skierowanego o nieujemnych wagach krawędzi.

Dane wejściowe

Dane wejściowe do testów stanowią losowo generowane grafy skierowane o zadanej liczbie wierzchołków i zadanej gęstości. Grafy są generowane automatycznie przez program, przy czym każda instancja grafu jest zapisywana do pliku tekstowego, z którego następnie wczytywane są dane do obu reprezentacji. Wagi krawędzi są losowane z ustalonego zakresu liczb całkowitych dodatnich: minimalna waga: 1-5, maksymalna waga: min+5 do min+25.

Środowisko testowe

Eksperyment jest wykonywany w środowisku konsolowym na komputerze z systemem Windows. Kod został napisany w języku C++. Wyniki mogą być zależne od wydajności sprzętu, dlatego wnioski dotyczące efektywności algorytmów są formułowane w sposób ogólny.

Specyfikacja sprzętowa

Laptop: ASUS TUF Gaming F15.

Procesor: Intel Core i5-11400H, 6 rdzeni, 12 wątków, taktowanie bazowe 2.7 GHz (do 4.5 GHz w trybie turbo).

Pamięć RAM: 16 GB DDR4.

Dysk: SSD NVMe 512 GB (Samsung MZVLQ512HBLU-00B00).

Karta graficzna: NVIDIA GeForce RTX 3050 Laptop GPU oraz Intel UHD Graphics.

System operacyjny: Microsoft Windows 11 Home 64-bit, wersja 10.0.26100.

Oczekiwane wyniki (hipotezy)

- **H1** - Dla grafów rzadkich (niewielka gęstość) algorytm Dijkstry na liście sąsiedztwa będzie działał szybciej niż na macierzy sąsiedztwa.
- **H2** - Dla grafów gęstych (duża gęstość) różnica w czasie działania obu wersji algorytmu będzie się zmniejszać.
- **H3** - Wraz ze wzrostem liczby wierzchołków i gęstości grafu czas działania algorytmu będzie rosnąć.

4. Implementacja

Program umożliwia interaktywną obsługę poprzez menu konsolowe. Struktura kodu została podzielona na funkcje odpowiedzialne za generowanie grafów, wczytywanie danych, prezentację grafu w różnych reprezentacjach oraz uruchamianie i testowanie algorytmu Dijkstry. Poniżej opisano najważniejsze elementy implementacji oraz funkcje dostępne w menu programu.

Struktura programu

- **open_file** - Funkcja pomocnicza do otwierania pliku tekstowego o podanej nazwie. Przyjmuje nazwę pliku i zwraca wartość logiczną **true** jeśli plik został otwarty poprawnie, w przeciwnym razie wypisuje komunikat o błędzie i zwraca **false**.
- **load_file** - Pozwala użytkownikowi wczytać graf z pliku. Wysyła zapytanie o nazwę pliku, sprawdza czy plik istnieje i można go otworzyć. Jeśli tak, ustawia podaną nazwę. W przeciwnym razie informuje o błędzie.
- **generate_graph** - Generuje losowy graf skierowany o zadanej liczbie wierzchołków i gęstości. Oblicza liczbę krawędzi, losuje wagi krawędzi, unika pętli i duplikatów. Następnie zapisuje graf do pliku graf.txt.
- **list** - Wczytuje graf z pliku do reprezentacji jako lista sąsiedztwa (tablica wektorów par). Odczytuje liczbę wierzchołków i krawędzi, wczytuje wszystkie krawędzie do odpowiednich pozycji.
- **matrix** - Wczytuje graf z pliku do reprezentacji jako macierz sąsiedztwa (tablica dwuwymiarowa). Odczytuje liczbę wierzchołków i krawędzi, wczytuje wszystkie krawędzie i ustawia wartości na odpowiednich pozycjach.
- **dijkstra_list** - Wykonuje algorytm Dijkstry na grafie wczytanym jako lista sąsiedztwa. Wczytuje graf, sprawdza poprawność wierzchołków, inicjalizuje tablice odległości, poprzedników i odwiedzin. Używa kolejki priorytetowej do wyboru najbliższego wierzchołka. Aktualizuje odległości i poprzedników dla sąsiadów.
- **dijkstra_matrix** - Wykonuje algorytm Dijkstry na grafie wczytanym jako macierz sąsiedztwa. Wczytuje graf, sprawdza poprawność wierzchołków, inicjalizuje tablice odległości, poprzedników i odwiedzin. W każdej iteracji wybiera nieodwiedzonego wierzchołka o najmniejszej odległości i aktualizuje odległości dla wszystkich sąsiadów.
- **main** - Główna funkcja programu obsługująca menu i logikę sterującą. Inicjalizuje parametry testowe (tablice n i density). Wyświetla menu i pobiera wybór użytkownika.

Opcje w menu programu

0. Wyjście - zakończenie działania programu.

1. Generuj losowy graf - Umożliwia użytkownikowi wygenerowanie nowego losowego grafu o zadanej liczbie wierzchołków i gęstości. Po wygenerowaniu graf zostaje zapisany do pliku tekstowego.

2. Wczytaj plik z grafem - Pozwala na wczytanie grafu z pliku wskazanego przez użytkownika. Program sprawdza, czy plik istnieje i czy można go otworzyć. Po poprawnym wczytaniu plik staje się aktualnym grafem.

3. Wyświetl graf (lista i macierz) - Wyświetla aktualny graf w obu reprezentacjach: jako listę sąsiedztwa oraz jako macierz sąsiedztwa. Pozwala to użytkownikowi zweryfikować poprawność wczytanych lub wygenerowanych danych.

4. Dijkstra (lista i macierz) - Umożliwia uruchomienie algorytmu Dijkstry dla aktualnego grafu. Użytkownik podaje wierzchołek początkowy i końcowy, a program wyznacza najkrótszą ścieżkę i jej długość zarówno dla reprezentacji listowej, jak i macierzowej. Dodatkowo mierzony jest czas działania algorytmu dla obu wersji.

5. Testowanie - Automatycznie przeprowadza serię testów wydajnościowych algorytmu Dijkstry dla różnych rozmiarów i gęstości grafu. Dla każdej konfiguracji grafu algorytm uruchamiany jest wielokrotnie, a wyniki są uśredniane i prezentowane w konsoli.

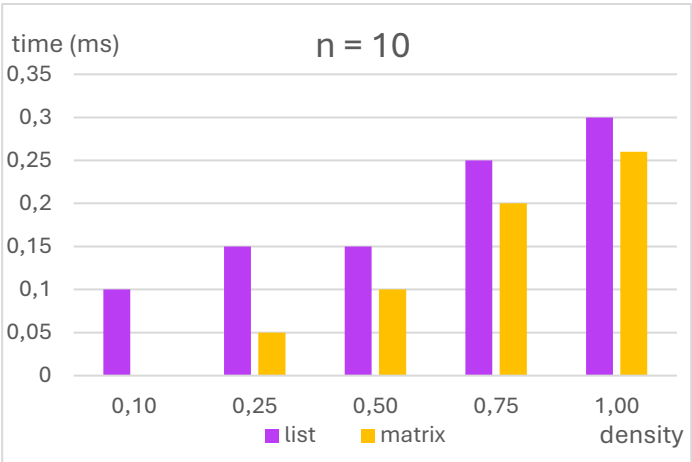
Uwagi

- Kod został odpowiednio skomentowany, aby ułatwić jego zrozumienie i analizę.
- Program dba o zwalnianie dynamicznie alokowanych struktur danych (listy i macierze sąsiedztwa) po zakończeniu pracy z grafem, co zapobiega wyciekom pamięci.
- Program sprawdza poprawność otwarcia plików oraz zakresy podanych wierzchołków, co zwiększa niezawodność działania.
- **WAŻNE:** Jeśli użytkownik nie wczyta wcześniej żadnego grafu z pliku (opcja 2), to przy wyborze opcji **3** lub **4** program automatycznie generuje losowy graf o domyślnych parametrach (liczba wierzchołków i gęstość z pierwszego elementu tablic `n` i `density`). Wygenerowany graf jest zapisywany do pliku `graf.txt` i staje się aktualnym grafem do dalszych operacji.

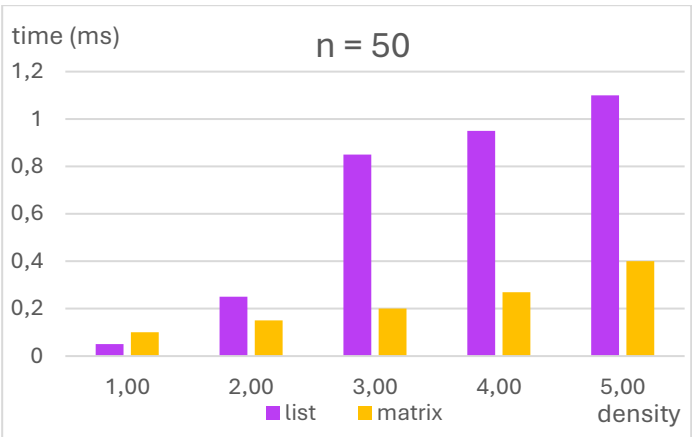
5. Zestawienie wyników

W ramach eksperymentu przeprowadzono pomiary czasu działania algorytmu Dijkstry dla obu reprezentacji grafu (lista sąsiedztwa i macierz sąsiedztwa) dla pięciu różnych rozmiarów grafu: **`n = 10, 50, 100, 500`** oraz **`1000`**. Dla każdego rozmiaru grafu testowano pięć poziomów gęstości: **`density = 0.10, 0.25, 0.50, 0.75`** oraz **`1.00`**. Każdy pomiar był uśredniony z 20 powtórzeń, a wierzchołki startowy i końcowy były losowane dla każdego testu. Dla każdego zestawu danych był generowany inny (losowy) graf.

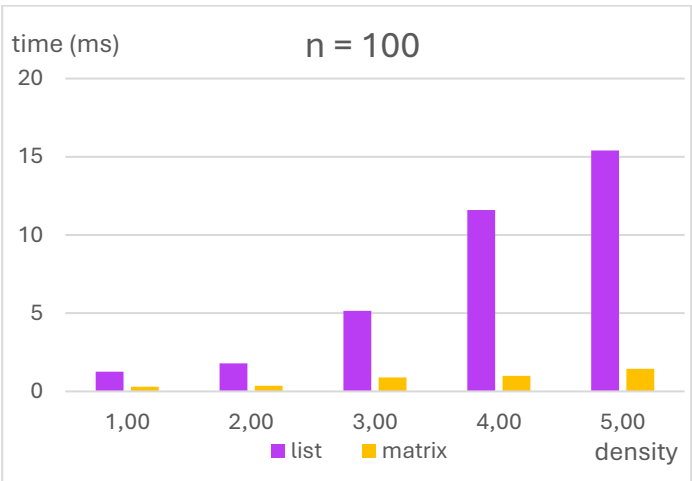
n = 10		
density	list	matrix
0,10	0,1	0
0,25	0,15	0,05
0,50	0,15	0,1
0,75	0,25	0,2
1,00	0,3	0,26



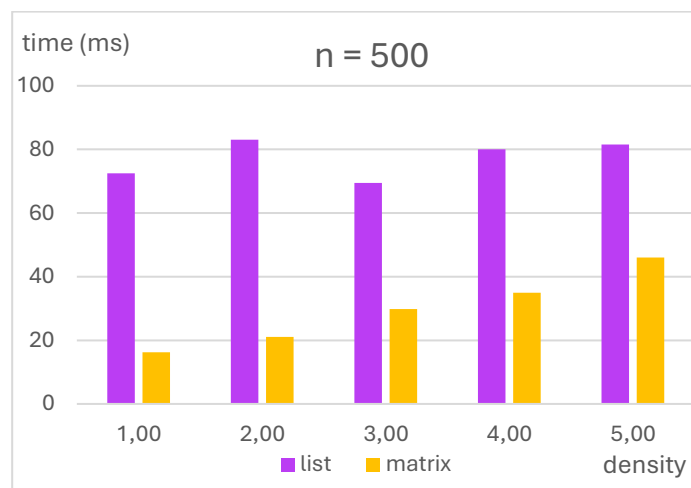
n = 50		
density	list	matrix
0,10	0,05	0,1
0,25	0,25	0,15
0,50	0,85	0,2
0,75	0,95	0,27
1,00	1,1	0,4



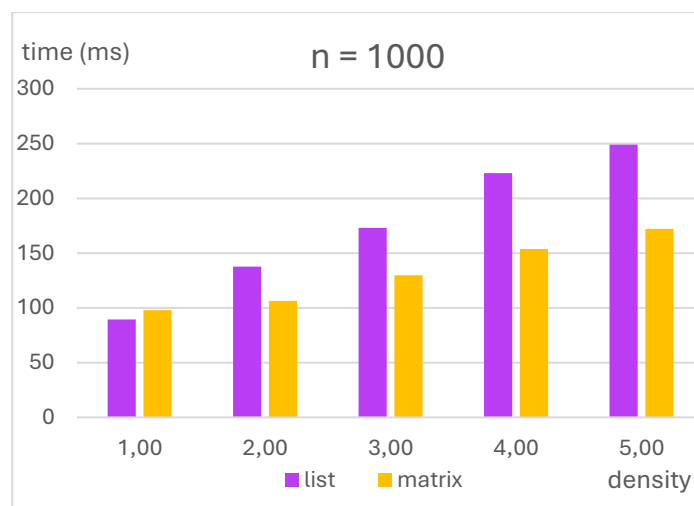
n = 100		
density	list	matrix
0,10	1,25	0,3
0,25	1,8	0,35
0,50	5,15	0,9
0,75	11,6	1
1,00	15,4	1,45



n = 500		
density	list	matrix
0,10	72,45	16,25
0,25	83,1	21,05
0,50	69,5	29,8
0,75	80,05	35
1,00	81,6	46,05



n = 1000		
density	list	matrix
0,10	89,3	97,8
0,25	137,65	106,4
0,50	172,95	129,6
0,75	222,85	153,8
1,00	249,05	172,2



Porównanie wyników

Dla listy sąsiedztwa czasy działania algorytmu Dijkstry rosną wraz ze wzrostem liczby wierzchołków oraz gęstości grafu. Dla najmniejszych grafów ($n = 10, 50$) czasy są bardzo krótkie (rzędu dziesiątych części milisekundy), natomiast dla większych grafów ($n = 500, 1000$) czas działania algorytmu wzrasta do kilkudziesięciu lub nawet ponad dwustu milisekund przy największych gęstościach. Widać wyraźny wpływ zarówno liczby wierzchołków, jak i gęstości na wydłużenie czasu działania algorytmu. Porównując oba typy reprezentacji, można zauważyć, że dla małych grafów ($n = 10, 50, 100$) macierz sąsiedztwa pozwala na uzyskanie krótszych czasów działania algorytmu niż lista sąsiedztwa, niezależnie od gęstości. Dla większych grafów ($n = 500, 1000$) różnice w czasie działania obu wersji algorytmu stają się mniejsze, jednak macierz sąsiedztwa nadal często wypada korzystniej, zwłaszcza przy większych gęstościach. W obu reprezentacjach wzrost gęstości grafu powoduje wydłużenie czasu działania algorytmu Dijkstry. Efekt ten jest szczególnie widoczny dla większych wartości n , gdzie różnice czasów pomiędzy najniższą a najwyższą gęstością są bardzo wyraźne.

6. Wnioski

W trakcie realizacji projektu w pełni zrealizowano wszystkie założone cele. Przede wszystkim udało się zaimplementować algorytm Dijkstry zarówno dla reprezentacji listowej, jak i macierzowej grafu w języku C++. Opracowany program umożliwia generowanie losowych grafów o zadanych parametrach, ich wczytywanie, prezentację w obu reprezentacjach oraz przeprowadzanie pomiarów czasu działania algorytmu. Dzięki temu możliwe było przeprowadzenie szerokiej serii eksperymentów dla różnych rozmiarów i gęstości grafów, a także zebranie i analiza wyników. W odniesieniu do celu dotyczącego implementacji algorytmu Dijkstry dla obu reprezentacji, program działa poprawnie i pozwala na łatwe porównywanie wydajności obu podejść. Kod został napisany w sposób modularny, co umożliwia dalszą rozbudowę i modyfikację. Cel związany z przeprowadzeniem eksperymentów dla grafów o różnych rozmiarach i gęstościach został zrealizowany poprzez automatyczne generowanie grafów oraz wielokrotne uruchamianie algorytmu dla każdej konfiguracji. Dzięki temu uzyskano wiarygodne, uśrednione wyniki, które pozwalają na rzetelną analizę wpływu parametrów grafu na czas działania algorytmu. Ostatni cel, czyli analiza wyników i sformułowanie wniosków dotyczących efektywności obu podejść, został osiągnięty poprzez szczegółowe zestawienie i porównanie czasów działania algorytmu Dijkstry w obu reprezentacjach. Analiza wyników pozwoliła na ocenę, w jakich przypadkach dana reprezentacja grafu jest bardziej efektywna.

Analizując zebrane dane, można odnieść się do postawionych hipotez:

H1: Wyniki eksperymentów nie potwierdziły tej hipotezy w pełni. W większości przypadków, zwłaszcza dla małych i średnich grafów, algorytm Dijkstry na macierzy sąsiedztwa działał szybciej niż na liście sąsiedztwa, nawet przy niskich gęstościach. Może to wynikać z lepszej lokalności pamięci macierzy oraz narzutu związanego z obsługą dynamicznych struktur danych w przypadku listy sąsiedztwa. Przewaga listy sąsiedztwa nie ujawniła się nawet dla największych testowanych grafów i najniższych gęstości.

H2: Ta hipoteza została potwierdzona. Wraz ze wzrostem gęstości grafu czasy działania obu wersji algorytmu zbliżają się do siebie. Dla bardzo gęstych grafów (density = 1.0) różnice czasowe pomiędzy reprezentacjami są stosunkowo mniejsze.

H3: Hipoteza ta została jednoznacznie potwierdzona. Zarówno wzrost liczby wierzchołków, jak i gęstości grafu powoduje wyraźny wzrost czasu działania algorytmu Dijkstry w obu reprezentacjach. Efekt ten jest szczególnie widoczny dla dużych grafów i wysokich gęstości, gdzie czasy działania rosną nawet kilkakrotnie w stosunku do grafów mniejszych lub rzadszych.

7. Podsumowanie

Projekt pozwolił na praktyczną weryfikację teoretycznych przewidywań dotyczących wydajności algorytmu Dijkstry w różnych reprezentacjach grafu. Osiągnięto wszystkie cele, a uzyskane wyniki dostarczyły cennych informacji na temat efektywności obu podejść w zależności od charakterystyki grafu. Warto zauważyć, że rzeczywista wydajność może być silnie uzależniona od szczegółów implementacyjnych oraz środowiska sprzętowego, co zostało uwzględnione w analizie.

8. Literatura

- https://pl.wikipedia.org/wiki/Algorytm_Dijkstry
- [https://en.wikipedia.org/wiki/Graph_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Graph_(abstract_data_type))
- <https://en.cppreference.com/w/cpp/container/vector>
- https://en.cppreference.com/w/cpp/container/priority_queue